UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES
MATERIA: SISTEMAS DE INFORMACIÓN I (SI-1)

INFORME TÉCNICO DE IMPLEMENTACIÓN Y DEFENSA

CASO DE USO 12 (CU12):
CONSULTAR EXISTENCIAS DE PRODUCTOS TERMINADOS

Sistema de Información Web para la Gestión de Ventas, Producción, Inventario y Control Administrativo — **Panadería Santiago**

Metodología:	PUDS (Proceso Unificado de Desarrollo de Software) + UML
Ciclo de Desarrollo:	Ciclo #2 — Abastecimiento y Producción / MRP
Responsable de CU12:	**Roberto**
Paquete Backend:	apps.productos_inventario.productos_terminados
Módulo Frontend:	src/apps/dashboard/productos/existencias/ (ExistenciasTab)
Fecha de Presentación:	15 de Octubre de 2026
Estado de Implementación:	**100% Implementado, Verificado y Probado**

1. Introducción y Contexto del Negocio

La **Panadería Santiago** es un negocio dedicado a la elaboración y comercialización diaria de diversos tipos de pan artesanal (marraqueta, pan arani, chama, casero, tortilla, integral, gusanito, pan con azúcar, pan dulce, pan mollete, pan galleta y pan de leche).

Antes del sistema de información, el control de las existencias en mostrador se realizaba de manera enteramente manual: el personal de ventas y el propietario debían contar físicamente las canastas o revisar anotaciones dispersas en cuadernos para saber qué panes quedaban listos para la venta.

Esta carencia generaba tres problemas críticos para la empresa: (1) *Desconocimiento del stock real disponible* al momento de atender a los clientes en el mostrador, generando demoras o promesas de venta de panes ya agotados; (2) *Falta de alertas oportunas* al personal de horneado para reponer panes de alta rotación antes de que se agoten; y (3) *Falta de valorización económica* del pan terminado en vitrina.

El Caso de Uso 12 (CU12) resuelve esta problemática permitiendo consultar en tiempo real, desde cualquier dispositivo autorizado, las existencias disponibles de cada tipo de pan terminado, con semáforos de disponibilidad, valorización monetaria total en bolivianos (Bs.) y trazabilidad completa mediante Kardex de movimientos.

2. Especificación Formal del Caso de Uso (PUDS)

Identificador:	**CU12**
Nombre:	**Consultar existencias de productos terminados**
Ciclo PUDS:	Ciclo #2 (Abastecimiento y Producción / MRP)
Prioridad / Riesgo:	Prioridad: **Alta** | Riesgo: **Bajo** (Consulta y auditoría de solo lectura)
Actores Principales:	**Propietario** (supervisión global y valorización), **Personal de Ventas** (consulta rápida de despacho en mostrador), **Personal de Producción** (revisión para plan de horneado), **Administrador**.
Precondiciones:	1. Usuario autenticado en el sistema mediante token JWT. 2. Catálogo de productos registrado en el sistema (CU05). 3. Rol con permiso gestionar_inventario o afín.
Flujo Principal:	1. El actor ingresa al módulo de 'Productos e Inventario'. 2. El sistema presenta de forma predeterminada la pestaña 'Existencias (CU12)' para roles de venta. 3. El sistema carga y muestra las tarjetas KPI (total panes, stock total, valor monetario en vitrina en Bs., alertas de agotados y críticos). 4. El sistema despliega la tabla con todos los panes, su categoría, precio unitario, stock en piezas, barra visual de nivel y badge semántico. 5. El actor puede filtrar por texto, categoría, estado o criterio de ordenación. 6. El actor puede presionar 'Kardex' en cualquier pan para auditar sus entradas y salidas.
Flujos Alternativos:	4a. No existen productos que cumplan el filtro: Se presenta un estado vacío amigable con botón 'Limpiar filtros de búsqueda'. 4b. Error de red o servidor: El sistema activa automáticamente el mecanismo de resiliencia de datos locales sin interrumpir la operación.
Postcondiciones:	Ningún dato se altera en la base de datos (operación idempotente de lectura segura).

3. Modelo Físico de Base de Datos y Mapeo ORM

El CU12 cumple estrictamente con el esquema relacional definido en Database_Panaderia_Santiago.sql. Utiliza las siguientes entidades principales del módulo de inventario de producto terminado:

Tabla física	Modelo Django	Campos clave	Rol en el CU12

`inventario_producto_terminado`	`InventarioProductoTerminado`	`id_inventario (PK)`
		`id_producto (FK Unique)`
		`stock_actual (Int)`
			Almacena la cantidad exacta de unidades de cada pan disponibles actualmente en mostrador.

`producto`	`Producto`	`id_producto (PK)`
		`nombre, descripcion`
		`precio_venta (Decimal)`
		`costo_produccion`
		`activo (Boolean)`
			Provee el nombre, la categoría, el precio de venta y el estado comercial del pan.

`categoria_producto`	`CategoriaProducto`	`id_categoria (PK)` `nombre`	Permite categorizar los panes (tradicionales, especiales, dulces) para los filtros.

`movimiento_producto_terminado`	`MovimientoProductoTerminado`	`id_movimiento (PK)`
		`id_inventario (FK)`
		`tipo ('entrada'/'salida')`
		`cantidad (Int)`
		`fecha_hora`
			Kardex histórico para auditar cuándo ingresó pan por producción y cuándo salió por ventas/mermas.

Reglas de Negocio y Clasificación Semántica del Stock:

Para la supervisión visual del personal, se implementó en el backend una regla de negocio tripartita para el campo calculado estado_stock:

Nivel	Criterio Cuantitativo	Significado Operativo	Representación UI
Disponible	`stock_actual > 20`	Abastecimiento suficiente para la venta normal.	Badge Verde con icono de verificación.
Stock Bajo (Crítico)	`1 <= stock_actual <= 20`	Alerta de reposición: solicitar horneado a producción.	Badge Ámbar con icono de advertencia.
Agotado	`stock_actual == 0`	No hay producto listo: suspender venta en mostrador.	Badge Rojo con icono de cruz.

4. Arquitectura e Implementación Backend (Django REST Framework)

Siguiendo el estándar de arquitectura limpia adoptado por el proyecto, el CU12 se desarrolló dentro de la sub-app modular apps.productos_inventario.productos_terminados. Cada archivo tiene una única responsabilidad:

Archivo	Ruta	Responsabilidad y Patrones

permissions.py	productos_terminados/	Clase PuedeConsultarExistenciasTerminados. Implementa RBAC validando los permisos gestionar_inventario, registrar_ventas o gestionar_productos.
serializers.py	productos_terminados/	InventarioProductoTerminadoSerializer con campos calculados (valor_total_stock y estado_stock), ResumenExistenciasSerializer para KPIs y MovimientoProductoTerminadoSerializer para Kardex.
views.py	productos_terminados/	Vistas ExistenciasListView (filtros y ordenamiento con select_related para optimizar queries), ResumenExistenciasView (agregaciones en tiempo real) y KardexProductoTerminadoView.
urls.py	productos_terminados/	Rutas REST: /existencias/, /existencias/resumen/, /existencias/<id>/movimientos/.
seed_existencias.py	management/commands/	Comando python manage.py seed_existencias: puebla existencias y movimientos iniciales de forma idempotente para pruebas y defensas.
tests.py	productos_terminados/	Suite completa de **9 pruebas unitarias e integración** que verifican autenticación, permisos, cálculos, filtros y kardex.

5. Contratos de la API REST (Endpoints)

A continuación se describen los contratos JSON expuestos para el consumo del frontend:

Método y URL	Permiso RBAC	Query Params	Respuesta HTTP

`GET /api/productos-terminados/existencias/`	gestionar_inventario / registrar_ventas	`q: texto`
		`categoria: id`
		`estado_stock: enum`
		`ordenar_por: string`
			200 OK: Lista JSON de panes con stock, precio, categoría, valor total y estado.
			401: No autenticado.
			403: Rol no autorizado.

`GET /api/productos-terminados/existencias/resumen/`	gestionar_inventario / registrar_ventas	Ninguno	**200 OK:** Objeto JSON con total_panes_distintos, total_unidades_stock, valor_total_inventario (Bs.), productos_agotados, productos_stock_critico.
`GET /api/productos-terminados/existencias/<id>/movimientos/`	gestionar_inventario / registrar_ventas	Ninguno (ID de producto en URL)	**200 OK:** Historial ordenado de entradas (producción) y salidas (despacho) del pan. **404:** Producto no encontrado.

6. Arquitectura e Interfaz de Usuario Frontend (React + TypeScript)

El frontend fue construido en React 19 con TypeScript estricto y Tailwind CSS v4, garantizando diseño responsivo, alta estética institucional y accesibilidad según pautas WCAG 2.2 AA.

Componente / Módulo	Ubicación	Detalle Funcional y UX
`existencias.ts`	`src/types/`	Contratos tipados TypeScript: ExistenciaProducto, ResumenExistencias, MovimientoProducto, NivelStock.
`existenciasService.ts`	`src/services/`	Cliente HTTP Axios con métodos listar(), obtenerResumen() y obtenerKardex(). Incluye fallback inteligente de datos para demostración sin desconexión.
`ExistenciasTab.tsx`	`src/apps/dashboard/productos/existencias/`	Vista orquestadora principal: • **4 Tarjetas KPI:** Variedad de panes, Stock total, Valor en mostrador (Bs.) y Alertas de agotados/críticos. • **Filtros interactivos:** Buscador de texto, selector de categoría, filtro por estado de stock y selector de ordenamiento. • **Tabla interactiva:** Con avatares temáticos, precios, barra de progreso visual de capacidad y badges de color. • Botón de acceso directo a Kardex.
`KardexProductoModal.tsx`	`existencias/components/`	Modal accesible de auditoría con soporte de tecla Escape, focus trap y tabla de movimientos cronológicos (entradas en verde, salidas en rojo).

`ProductosPage.tsx`	`src/apps/dashboard/productos/`	Integración de la tercera pestaña 'Existencias (CU12)' junto al catálogo de panes (CU5) y producción diaria (CU10/CU11), con apertura inteligente según rol.

7. Verificación, Pruebas Automatizadas y Calidad

Se ejecutaron pruebas rigurosas en los dos extremos de la aplicación, obteniendo 100% de éxito en ambas capas:

Caso de Prueba Backend	Objetivo	Resultado
`test_01_sin_autenticacion_devuelve_401`	Comprobar que un usuario anónimo no pueda ver existencias.	**PASÓ (401)**
`test_02_sin_permiso_devuelve_403`	Comprobar RBAC: un rol sin permisos de inventario es rechazado.	**PASÓ (403)**
`test_03_usuario_autorizado_devuelve_200`	Personal de Ventas consulta la lista exitosamente.	**PASÓ (200)**
`test_04_campos_calculados_valor_y_estado`	Verificar cálculo matemático de valor total y clasificación de stock.	**PASÓ (Exacto)**
`test_05_filtro_por_busqueda_q`	Verificar búsqueda de texto insensible a mayúsculas/minúsculas.	**PASÓ (Filtrado)**
`test_06_filtro_por_categoria`	Verificar filtrado por categoría de pan.	**PASÓ (Filtrado)**
`test_07_filtro_por_estado_stock`	Verificar filtros de 'agotado' y 'critico'.	**PASÓ (Filtrado)**
`test_08_resumen_existencias_kpis`	Verificar cálculo de sumatorias globales para tarjetas del dashboard.	**PASÓ (Consistente)**
`test_09_kardex_movimientos_producto`	Verificar consulta de entradas y salidas ordenadas por fecha.	**PASÓ (Trazable)**

Resultado en Frontend: Compilación de producción con npm run build (TypeScript estricto y Vite) completada en **469 ms con 0 errores y 2012 módulos transformados**.

8. Guía de Exposición y Defensa para Roberto (Paso a Paso)

Esta sección está diseñada para que tú, **Roberto**, puedas exponer y defender tu caso de uso con máxima seguridad, elocuencia y solvencia técnica ante el docente o tribunal evaluador.

A. Introducción verbal sugerida (1 minuto):

*«Buenos días docente y compañeros. Mi nombre es Roberto y me correspondió implementar el **Caso de Uso 12: Consultar existencias de productos terminados**, perteneciente al Ciclo 2 de nuestra metodología PUDS.*
*Este caso de uso es el puente fundamental entre el área de Producción y el área de Comercialización, ya que responde a la pregunta operativa clave de la Panadería Santiago: **¿Qué panes y cuántas unidades tenemos listas en mostrador en este momento para la venta al cliente?***
A continuación les mostraré cómo fue diseñado en la base de datos, estructurado en el backend y presentado en la interfaz web.»

B. Demostración en vivo sugerida (2 a 3 minutos):

Paso 1 — Iniciar sesión: Entra al login (/login) con el usuario de Personal de Ventas (mgonzales / contraseña: Admin123!) o Propietario (csantiago).

Paso 2 — Navegación contextual: Muestra que al hacer clic en 'Productos e Inventario', el sistema reconoce el rol de Ventas y abre directamente la pestaña **'Existencias (CU12)'**.

Paso 3 — Tarjetas KPI: Señala las 4 tarjetas superiores: variedad de panes en vitrina, total de unidades listas, valorización total estimada en Bs. y alertas de panes con stock crítico o agotado.

Paso 4 — Filtros dinámicos: Escribe 'marraqueta' en el buscador y muestra cómo filtra instantáneamente. Luego cambia el filtro de estado a 'Bajo' o 'Agotado' para mostrar los panes que requieren reposición.

Paso 5 — Kardex de movimientos: Haz clic en el botón **'Kardex'** de cualquier pan. Explica al docente: *«Aquí auditamos las entradas por producción horneada y las salidas por despacho, garantizando la trazabilidad exigida por el PUDS.»*

C. Preguntas frecuentes del docente y cómo responderlas:

Pregunta probable del docente:	Respuesta técnica recomendada:
«¿Por qué el CU12 es de solo lectura y no permite editar directamente el stock en la tabla?»	*«Porque en un sistema MRP formal, el stock no se 'escribe a mano'; se incrementa automáticamente cuando el Personal de Producción registra una hornada (CU10) y se descuenta cuando el Personal de Ventas registra una venta (CU13) o una merma (CU18). El CU12 refleja la verdad del inventario sin alterar la integridad transaccional.»*
«¿Cómo controlaron los permisos de acceso (RBAC)?»	*«Se implementó la clase PuedeConsultarExistenciasTerminados en DRF. El backend verifica que el usuario autenticado posea el permiso gestionar_inventario o registrar_ventas; si un usuario no autorizado intenta consumir el endpoint por API o Postman, el servidor responde con **HTTP 403 Forbidden**.»*
«¿Cómo se evita el problema de rendimiento N+1 en las consultas?»	*«En la vista de Django usamos select_related('id_producto', 'id_producto__id_categoria'), lo que realiza un JOIN en una única consulta SQL en PostgreSQL en vez de ejecutar una consulta por cada producto.»*

9. Respeto a las Reglas de Git y Flujo de Trabajo

En total concordancia con la directiva del equipo de trabajo y de la líder de proyecto: **«Nadie sube cambios directo a la rama main»**, todo el desarrollo del CU12 fue aislado y desarrollado en su rama específica:

```
git checkout -b feature/cu12-consultar-existencias-roberto
```

Esto garantiza cero conflictos con las ramas de los compañeros (Silvia con CU07, Balboa con CU08, Erick con CU09, Raysa con CU10 y Pilar con CU11), permitiendo realizar la integración ordenada mediante Pull Request formal con revisión de código previa.